

Hadoop Installation & Setup Guide

Single-Node Hadoop Cluster on Windows 11 via WSL2 (Ubuntu 22.04)

This document walks through every step you followed to install and configure Apache Hadoop in a single-node ('pseudo-distributed') setup inside WSL2, and — more importantly — explains **why** each step exists and the **core distributed-systems concept** behind it. It is meant as both a record of what was done and a study guide for understanding Hadoop's architecture.

Component	Version
Operating System	Windows 11
WSL	2.7.11
Linux Distribution	Ubuntu 22.04 LTS
Java	OpenJDK 11
Apache Hadoop	3.3.6

What is Hadoop, in one paragraph?

Apache Hadoop is a framework for storing and processing very large datasets across clusters of ordinary computers. It has two core pillars: **HDFS** (Hadoop Distributed File System), which splits and replicates files across machines so storage can scale horizontally and survive hardware failure, and **YARN** (Yet Another Resource Negotiator), which schedules and manages the compute resources (CPU/memory) that processing jobs such as MapReduce need. In this guide, Hadoop runs in **pseudo-distributed mode**: every daemon (NameNode, DataNode, ResourceManager, etc.) runs on the single WSL2 machine, simulating a cluster for learning and development purposes.

1. Why WSL2 Instead of Native Windows

Hadoop is built for Unix-like environments: its shell scripts, file permission model, and native libraries assume Linux (or at least a POSIX-compliant OS). Running Hadoop natively on Windows requires special native binaries (winutils.exe) and is fragile and poorly supported. WSL2 solves this by running a real, lightweight Linux kernel inside Windows, so Hadoop, SSH, and all its shell tooling behave exactly as they would on a genuine Linux server — while you still get to keep Windows as your daily desktop OS.

■ Core concept: WSL2 vs. WSL1

WSL2 runs a real Linux kernel inside a lightweight Hyper-V virtual machine, giving full system-call compatibility, whereas WSL1 translated Linux calls to Windows calls and broke on things like proper socket/SSH behavior. Hadoop needs full POSIX semantics, so WSL2 is the correct choice.

STEP 1 — Install WSL

```
wsl --install

# if corrupted:
winget install Microsoft.WSL

wsl --version
```

What you did: Installed the Windows Subsystem for Linux platform itself (the virtualization layer and management tooling), and repaired it via winget when the install was corrupted.

Why this step: WSL must exist before any Linux distribution can be installed on top of it — it is the engine that hosts the Ubuntu virtual environment used for every subsequent step.

STEP 2 — Install Ubuntu 22.04

```
wsl --list --online
wsl --install -d Ubuntu-22.04
```

What you did: Installed a specific Linux distribution (Ubuntu 22.04 LTS) inside WSL2 and created a Linux user account and password for it.

Why this step: Ubuntu 22.04 LTS is a stable, well-supported release with long-term security updates, and it is one of the most common environments Hadoop tutorials and production deployments target, minimizing the chance of compatibility surprises.

STEP 3 — Update Ubuntu

```
sudo apt update
sudo apt upgrade -y
```

What you did: Refreshed the local package index (apt update) and then upgraded all installed packages to their latest available versions (apt upgrade).

Why this step: A fresh Ubuntu image can have an outdated package list and security patches. Updating first avoids dependency conflicts and missing-package errors later when installing Java, SSH, and other tools.

STEP 4 — Install Java 11

```
sudo apt install openjdk-11-jdk openssh-server rsync wget curl -y

java -version
```

What you did: Installed the Java Development Kit (OpenJDK 11), the OpenSSH server, plus rsync, wget, and curl, and confirmed the Java version.

Why this step: Hadoop itself is written in Java and runs as a set of Java Virtual Machine (JVM) processes — it cannot start without a JDK present. Hadoop 3.3.x officially supports and is most stable on Java 8 or 11. SSH, rsync, wget, and curl are utilities Hadoop's own start/stop scripts rely on: SSH to launch daemons (even locally), and wget/curl to fetch the Hadoop distribution.

■ Core concept: Why Hadoop needs Java

HDFS, YARN, and MapReduce daemons (NameNode, DataNode, ResourceManager, NodeManager) are all long-running JVM processes. The JVM gives Hadoop platform independence and manages memory for these daemons, which is why JAVA_HOME must be correctly configured for every Hadoop script to work.

2. Passwordless SSH — The Cluster Control Channel

STEP 5 — Configure SSH

```
ssh-keygen -t rsa -P ""

cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys

sudo service ssh start
ssh localhost # type: yes
exit
```

What you did: Generated a passwordless RSA key pair, added the public key to your own `authorized_keys` file, started the SSH daemon, and verified you could SSH into 'localhost' without being prompted for a password.

Why this step: Hadoop's `start-dfs.sh` / `start-yarn.sh` scripts use SSH to launch daemon processes on every node in the cluster — even when there is only one node, as here. If SSH required a password each time, the startup scripts would hang waiting for manual input. Passwordless key-based authentication lets Hadoop start (and stop) all daemons automatically with a single command.

■ Core concept: SSH as Hadoop's control plane

In a real multi-node cluster, the master node uses SSH to remotely start `DataNode` and `NodeManager` processes on every worker machine listed in the 'workers' file. On a single-node setup, 'localhost' plays the role of both master and worker, so Hadoop must still be able to SSH into itself without a password.

STEP 6 — Download Hadoop

```
cd ~
wget https://archive.apache.org/dist/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
tar -xzf hadoop-3.3.6.tar.gz
mv hadoop-3.3.6 hadoop
```

What you did: Downloaded the official Hadoop 3.3.6 binary distribution from the Apache archive, extracted the tarball, and renamed the folder to a simple 'hadoop' path.

Why this step: Downloading straight from the Apache archive guarantees an authentic, unmodified build. Renaming the folder to `~/hadoop` keeps all subsequent path references (`HADOOP_HOME`, config files) short and consistent.

3. Environment Variables — Telling Every Tool Where Things Live

STEP 7 — Configure Environment Variables

```
nano ~/.bashrc

# Java
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64

# Hadoop
export HADOOP_HOME=$HOME/hadoop
export HADOOP_INSTALL=$HADOOP_HOME
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export HADOOP_YARN_HOME=$HADOOP_HOME
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native

export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin

source ~/.bashrc
echo $JAVA_HOME
echo $HADOOP_HOME
```

What you did: Added `JAVA_HOME` and a family of `HADOOP_*_HOME` variables to your shell profile, added Hadoop's `bin/` and `sbin/` directories to `PATH`, then reloaded the profile so the current shell session picks up the new variables.

Why this step: Hadoop's internal scripts (`hadoop-config.sh` and friends) read these exact variable names to locate the JDK and the Hadoop installation itself. Without `JAVA_HOME`, Hadoop cannot launch its JVM processes at all. Adding `bin/` and `sbin/` to `PATH` lets you run commands like `hdfs`, `start-dfs.sh`, and `jps` from any directory instead of typing the full path every time.

■ Core concept: Environment variables as configuration discovery

Rather than hard-coding paths inside every script, Hadoop (like most JVM-based systems) relies on environment variables as a lightweight, shell-level configuration mechanism. This is the same pattern used by tools like Maven (`JAVA_HOME`), Android SDK (`ANDROID_HOME`), and Spark (`SPARK_HOME`).

STEP 8 — Configure `hadoop-env.sh`

```
nano ~/hadoop/etc/hadoop/hadoop-env.sh

# find:
# export JAVA_HOME=

# replace with:
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
```

What you did: Explicitly set `JAVA_HOME` a second time, but this time inside Hadoop's own environment script rather than the shell's `.bashrc`.

Why this step: When Hadoop daemons are started via SSH (as `start-dfs.sh` does), the remote shell session does not always source your interactive `.bashrc`, so `JAVA_HOME` set only there can be invisible to the daemon process. Setting it directly in `hadoop-env.sh` guarantees every Hadoop process — however it is launched — can find the JDK.

4. HDFS Storage Directories and Configuration

STEP 9 — Create Hadoop Storage Directories

```
mkdir -p ~/hadoopdata/hdfs/namenode
mkdir -p ~/hadoopdata/hdfs/datanode
```

What you did: Created two plain directories on the local Linux filesystem: one for NameNode metadata, one for DataNode block storage.

Why this step: HDFS needs real, local disk locations to persist its metadata (the namenode directory) and the actual data blocks (the datanode directory). These paths are referenced directly in `hdfs-site.xml` in the next step.

■ Core concept: NameNode vs. DataNode

HDFS splits every file into fixed-size blocks (128 MB by default) and distributes them across DataNodes. The NameNode is the 'librarian': it stores only metadata — the directory tree, filenames, permissions, and which blocks live on which DataNodes — never the file contents themselves. DataNodes are the 'warehouse workers': they physically store and serve the actual block data. Losing the NameNode's metadata makes the entire filesystem unreadable even if every block is intact, which is why namenode storage is treated so carefully in production (checkpoints, secondary NameNode, HA setups).

STEP 10 — Configure `core-site.xml`

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

What you did: Set the default filesystem URI that every Hadoop client and daemon uses to reach HDFS.

Why this step: `core-site.xml` is Hadoop's top-level, cross-cutting configuration file. `fs.defaultFS` tells every Hadoop command ('`hdfs dfs -ls /`', MapReduce jobs, etc.) that 'the filesystem' means HDFS running on localhost, port 9000 — rather than the local Linux filesystem or some other cluster.

STEP 11 — Configure hdfs-site.xml

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///home/nirob/hadoopdata/hdfs/namenode</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:///home/nirob/hadoopdata/hdfs/datanode</value>
  </property>
</configuration>
```

What you did: Set the HDFS block replication factor to 1, and pointed the NameNode and DataNode at the storage directories created in Step 9.

Why this step: In a production cluster, dfs.replication is usually 3 — each block is copied to three separate machines so the data survives disk or node failure. In a single-node lab setup there is only one DataNode, so replicating to 3 copies is impossible; setting it to 1 avoids constant 'under-replicated blocks' warnings while still letting you exercise the full HDFS/YARN workflow.

■ Core concept: Replication as fault tolerance

HDFS assumes hardware will fail. Instead of relying on expensive RAID arrays, it simply keeps multiple copies of every block on different machines (and ideally different racks). If one DataNode dies, the NameNode notices missing replicas via periodic heartbeats and instructs another DataNode to re-replicate the block, restoring the target replication factor automatically.

STEP 12 — Configure mapred-site.xml

```
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
  <property>
    <name>mapreduce.application.classpath</name>
    <value>${HADOOP_MAPRED_HOME}/share/hadoop/mapreduce/*:${HADOOP_MAPRED_HOME}/share/hadoop/mapreduce/lib
  </property>
</configuration>
```

What you did: Told MapReduce to run its jobs using YARN as the resource-management framework, and gave it the classpath needed to find its own jar files at runtime.

Why this step: Older versions of Hadoop (1.x) had MapReduce manage its own cluster resources directly. Since Hadoop 2.x, that responsibility was split out into YARN so other engines (Spark, Tez, etc.) could share the same cluster. This file tells Hadoop to use that modern model rather than a legacy local/standalone runner.

STEP 13 — Configure yarn-site.xml

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
  <property>
    <name>yarn.resourcemanager.hostname</name>
    <value>localhost</value>
  </property>
</configuration>
```

What you did: Enabled the 'shuffle' auxiliary service that MapReduce needs on every NodeManager, and told YARN where to find its ResourceManager (localhost, in this single-node case).

Why this step: The shuffle service is what moves intermediate data from Map tasks to Reduce tasks across the cluster; without registering it as an auxiliary service, MapReduce jobs fail during the shuffle phase. Setting resourcemanager.hostname lets every NodeManager know which machine is in charge of scheduling.

■ Core concept: YARN's two-level scheduler

YARN separates 'what to run' from 'how much capacity is available.' The ResourceManager (master) tracks total cluster resources and grants 'containers' (a CPU/memory allocation) to applications. Each NodeManager (worker) manages containers on its own machine and reports resource usage back to the ResourceManager. This generic resource model is what let YARN evolve from 'just runs MapReduce' into a general-purpose cluster operating system that also hosts Spark, Tez, Flink, and more.

5. Formatting, Starting, and Using HDFS

STEP 14 — Format the NameNode

```
hdfs namenode -format
```

What you did: Initialized a brand-new, empty HDFS filesystem: created the NameNode's metadata structures (fsimage, edit logs) inside the namenode directory configured earlier.

Why this step: HDFS cannot start until its metadata store exists in some valid initial state, similar to how a new hard disk must be formatted with a filesystem (ext4, NTFS, etc.) before it can store files. This is a one-time operation — running it again on an existing cluster wipes all metadata and effectively destroys the filesystem, so it is normally done only once.

STEP 15 — Start Hadoop

```
start-dfs.sh
start-yarn.sh
jps
```

What you did: Launched the HDFS daemons (NameNode, DataNode, SecondaryNameNode) and then the YARN daemons (ResourceManager, NodeManager), and used `jps` to confirm all Java processes were actually running.

Why this step: HDFS and YARN are independent subsystems with separate start scripts, which is why they are started (and stopped) individually rather than with one master command. `jps` (Java Process Status) is the quickest sanity check that every daemon actually launched instead of silently failing.

■ Core concept: What the Secondary NameNode actually does

Despite the name, the Secondary NameNode is not a hot standby / failover node. Its job is to periodically merge the NameNode's edit log into a new fsimage checkpoint, preventing the edit log from growing unbounded and speeding up NameNode restarts. True high availability requires a separate Standby NameNode setup, which is beyond this single-node lab configuration.

STEP 16 — Verify HDFS

```
hdfs dfs -mkdir /input
hdfs dfs -ls /
```

What you did: Created a directory directly inside HDFS and listed the HDFS root.

Why this step: This confirms the NameNode is actually accepting filesystem operations over its RPC port, not just that the process is running.

STEP 17 — Upload a File

```
echo "Hello Hadoop" > file1.txt
echo "This is a Hadoop Lab" >> file1.txt
```

```
hdfs dfs -put file1.txt /input
hdfs dfs -ls /input
hdfs dfs -cat /input/file1.txt
```

What you did: Created a small text file on the local Linux filesystem, then copied it into HDFS ('put'), listed it, and printed its contents back out ('cat') to prove the round trip works end to end.

Why this step: This is the full 'hello world' of HDFS: it demonstrates that a file handed to Hadoop is actually being chunked into blocks, written to the DataNode's storage directory, tracked by the NameNode, and retrievable on demand — the same mechanism that scales up to petabyte-sized files across thousands of machines.

6. Stopping and Restarting the Cluster

STEP 18 — Stop Hadoop

```
stop-yarn.sh  
stop-dfs.sh
```

Why this step: Cleanly stopping YARN before HDFS avoids leaving orphaned application processes and lets each daemon shut down gracefully, flushing any pending state to disk.

STEP 19 — Restart Hadoop (Next Time)

```
sudo service ssh start  
start-dfs.sh  
start-yarn.sh  
jps
```

Why this step: Unlike a native Linux install, WSL2 does not keep background services (like sshd) running automatically across a Windows reboot — they must be started manually each session. SSH must come up first since, as covered in Step 5, Hadoop's start scripts depend on it to launch every daemon.

7. Everyday HDFS Command Reference

These are the day-to-day commands for working with files once the cluster is running — conceptually similar to standard Linux file commands, but operating against HDFS instead of the local disk.

Command	Purpose
<code>hdfs dfs -mkdir /dir</code>	Create a directory in HDFS
<code>hdfs dfs -ls /</code>	List contents of an HDFS path
<code>hdfs dfs -put file.txt /dir</code>	Upload a local file into HDFS
<code>hdfs dfs -get /dir/file.txt</code>	Download a file from HDFS to local disk
<code>hdfs dfs -cat /dir/file.txt</code>	Print an HDFS file's contents
<code>hdfs dfs -cp /src /dst</code>	Copy within HDFS
<code>hdfs dfs -mv /src /dst</code>	Move / rename within HDFS
<code>hdfs dfs -rm /dir/file.txt</code>	Delete a file
<code>hdfs dfs -rm -r /dir</code>	Delete a directory recursively
<code>hdfs dfs -du /</code>	Show disk usage per path
<code>hdfs dfs -df /</code>	Show overall filesystem capacity

Verifying a Healthy Installation

Running `java -version`, `hadoop version`, and `jps` should together confirm the JDK, the Hadoop build, and — critically — that all six expected processes (NameNode, DataNode, SecondaryNameNode, ResourceManager, NodeManager, and the `jps` tool itself) are alive.

```
java -version
hadoop version
jps
```

Expected `jps` output:

```
NameNode
DataNode
SecondaryNameNode
ResourceManager
NodeManager
Jps
```

8. Operational Notes

On the native-library warning

The message `WARN util.NativeCodeLoader: Unable to load native-hadoop library... is expected on WSL`. Hadoop ships optional native (compiled C) libraries for performance-sensitive operations like compression; when they are missing, Hadoop transparently falls back to pure-Java implementations. Functionality is unaffected — only some operations may run marginally slower.

Shutdown discipline

Always stop YARN and then HDFS before shutting down the PC:

```
stop-yarn.sh
stop-dfs.sh
```

This lets the NameNode flush its edit log and each daemon exit cleanly, rather than being killed abruptly when Windows suspends or terminates the WSL2 VM.

Startup discipline after a reboot

Because WSL2 does not persist background services across a Windows restart, SSH must be started manually first, then the two Hadoop subsystems:

```
sudo service ssh start
start-dfs.sh
start-yarn.sh
```

Summary: How the Pieces Fit Together

WSL2 provides the Linux kernel Hadoop was designed for. Java provides the runtime every Hadoop daemon executes in. SSH lets Hadoop's scripts start and stop every daemon — even on a single machine pretending to be a cluster — without manual intervention. `core-site.xml`, `hdfs-site.xml`, `mapred-site.xml`, and `yarn-site.xml` progressively configure where the filesystem lives, how many copies of data to keep, and how compute resources are scheduled. Formatting the NameNode initializes an empty distributed filesystem, and starting HDFS and YARN brings the whole pseudo-cluster online — ready to store files reliably (HDFS) and run distributed computations against them (YARN/MapReduce), exactly as it would on a real multi-machine cluster, just scaled down to a single node for learning.

Environment reproduced: Windows 11 · WSL2 2.7.11 · Ubuntu 22.04 LTS · OpenJDK 11 · Apache Hadoop 3.3.6